AIBI Computer Vision for Manufacturing

– Team 9 –

Nathan DiGilio: Major in CS
Jacob Dzikowski: Major in CS
Collin Miller: Major in SWE
Daniel Pora: Major in SWE

Course Instructor: Naseem Ibrahim
Faculty Advisor: Jonathan Liaw

Industry Sponsor: AIBI Solutions
Industry Mentor: Kevin DiGilio

A capstone project report submitted to the faculty of
Computer Science and Software Engineering
Pennsylvania State University, Erie

October 2025
(Version 1.5)

Table of Contents

1. Abstract 2
2. Report Revision History 3
2.1. Changes in Version 1.5 4
3. Problem Statement 4
3.1. Business Background 5
3.2. Needs 5
3.3. Objectives 5
4. Initial Requirements 6
4.1. User Requirements 7
4.1.1. Glossary of Relevant Domain Terminology 7
4.1.2. User Groups 7
4.1.3. Functional Requirements 8
4.1.4. Non-functional Requirements 12
4.2. System Requirements 13
4.2.1. Functional Requirements 13
4.2.2. Non-functional Requirements 14
4.3. Requirements Trace Table 16
5. Exploratory Studies 16
5.1. Relevant Techniques 17
5.2. Relevant Packages/Products 17
5.3. Broader Impacts 17
6. System Design 18
6.1. Architectural Design 18
7. System Implementation 19
7.1. Programming Languages & Tools 20
7.2. Coding Conventions 21
7.3. Code Version Control 21
9. Challenges & Open Issues 21
9.1. Challenges Faced in Requirements Engineering 22
10. System Manuals 22
10.1. Instructions for System Development 23

12. References 24
13. Team Members' Log Sheets 25
13.1 Nathan DiGilio 26
13.2 Jacob Dzikowski 26
13.3 Daniel Pora 26
13.4 Collin Miller 27
Appendix U 28
1. Use Case Tables 29
Appendix R 35
1. User Functional Requirements: 36
2. User Non-functional Requirements: 41
3. System Functional Requirements 49
4. System Non-functional Requirements 56
5. Trace Table 70

1. Abstract

Manufacturing companies operating under strict compliance and government contract requirements face major challenges in maintaining traceability, efficiency, and data security. Many rely on outdated barcode systems or manual tracking, leading to inefficiencies and human error. To address these issues, AIBI Solutions, in collaboration with KMD Technology Solutions, is developing an on-premises computer vision system that automates barcode scanning and verifies manufacturing processes in real time. The system uses fixed cameras at each workstation to automatically detect and decode multiple barcodes, monitor process steps, and provide immediate feedback through an intuitive interface. Data is formatted into structured JSON outputs and integrated securely with KMD's existing workflow management system through a local API connector, ensuring full compatibility without cloud dependencies.

The project aims to increase production efficiency, reduce human error, and enhance operational transparency while maintaining data privacy and regulatory compliance. Beyond the immediate business value, the system demonstrates how artificial intelligence and computer vision can bring smart automation to secure, high stakes manufacturing environments, paving the way for broader industrial modernization.

This report documents the results of the requirements engineering and design process. It includes the project background and objectives, elicited user and system requirements, and implementation of details of the proposed computer vision solution.

2. Report Revision History

2.1. Changes in Version 1.5

- Fixed the Use Case Diagram
- Removed “Senior Design Student Developer” from Key Users
- Removed excessive references to “uv”
- Re-engineered system requirements so they can be testable
- Various grammatical improvements

3. Problem Statement

3.1. Business Background

AIBI Solutions was founded as an AI-focused sister company of KMD Technology Solutions, a company known for its project management software and workflow optimization tools. While KMD focused on improving digital coordination and project efficiency, AIBI was created to extend that mission into artificial intelligence, with a focus on secure, on-premises systems built for regulated industries.

The goal behind AIBI Solutions is to deliver advanced AI technology that can operate entirely within a company's local infrastructure, allowing manufacturers that work under government contracts or strict compliance regulations to take advantage of automation without compromising security. The company's first major product was a large language model–based quoting system designed to speed up and standardize the part quoting process. It reduced manual effort, increased consistency across quotes, and helped teams respond faster to customer requests while keeping all sensitive data stored on-site.

AIBI's next step is expanding into computer vision. This project focuses on building an intelligent tracking system that continuously monitors parts as they move through the facility, verifying that workers follow each step in the correct order. The goal is to bring greater transparency, efficiency, and accountability to manufacturing workflows, while maintaining strict compliance and ensuring that all processing happens locally, within the client's own secure environment.

3.2. Needs

Manufacturing companies that work under government contracts or strict compliance standards are under constant pressure to keep their production accurate, traceable, and fully accountable. Many still rely on manual tracking or outdated barcode systems that slow down operations and leave room for mistakes. At the same time, stricter data protection rules make it difficult to adopt cloud-based AI tools. These companies need solutions that can run completely on-premises, systems that deliver the speed and intelligence of modern AI without sending any data off-site.

AIBI Solutions was built to meet that need. The goal is to bring automation and insight into manufacturing environments that can't rely on traditional cloud infrastructure, combining large language models and computer vision to make production faster, safer, and more reliable.

3.3. Objectives

The objective of this phase is to develop a computer vision system that keeps track of parts and verifies that workers are completing each step in the correct order. The system will automatically monitor parts as they move through the facility, ensure each process follows the proper sequence, and alert supervisors when something is out of order. Everything will run locally to stay compliant with strict security requirements.

By doing this, the project aims to make manufacturing more transparent and efficient while reducing human error and maintaining full control of the data. The end goal is a reliable, on-premises AI solution that supports smarter decision-making and smoother production without adding more complexity to the workflow.

4. Initial Requirements

4.1. User Requirements

4.1.1. Glossary of Relevant Domain Terminology

- **AI:** Artificial Intelligence
- **API:** Application Programming Interfaces
- **GPU:** Graphical Processing Unit
- **JSON:** JavaScript Object Notation
- **LLM:** Large Language Model
- **QR:** Quick Response
- **REST:** Representational State Transfer
- **SQL:** Structured Query Language
- **UI:** User Interface

4.1.2. User Groups

1. Factory Operators (Key Users)

- **Role Responsibilities:** Operate machinery and handle parts during production. Use the vision system to scan codes and verify correct completion of each manufacturing step. Will respond to alerts when a deviation is detected by the system.
- **User Priority:** Key User - the system's success depends on their adoption and satisfaction.
- **Subject Matter Experience:** Master - extensive experience with manufacturing procedures and workflows.
- **Technological Experience:** Novice to Journeyman - varying comfort levels with technology; most are non-technical.
- **Other Characteristics:** Adult factory workers; may have limited time for training. Interfaces must be simple, visual, and accessible, supporting varying abilities and potential physical constraints.

2. Production Supervisors (Key Users)

- **Role Responsibilities:** Monitor workstation performance, review system logs, validate that steps are followed in sequence, and address reported process violations.
- **User Priority:** Key User - their input is essential for defining operational requirements and ensuring the system fits daily workflow oversight.
- **Subject Matter Experience:** Master - deep understanding of production flow and quality control.
- **Technological Experience:** Journeyman - familiar with digital dashboards and manufacturing data systems.
- **Other Characteristics:** Prefer concise, visual feedback and reliable alerts. May need multi-station access for monitoring.

3. AIBI Integration Engineers (Secondary Users)

- **Role Responsibilities:** Install, configure, and maintain the vision system; ensure integration with KMD Technology Solutions workflow management software.
- **User Priority:** Secondary User - critical during deployment and maintenance but less involved in daily use.
- **Subject Matter Experience:** Journeyman to Master - skilled in AI and computer vision systems.
- **Technological Experience:** Master - advanced technical skills
- **Other Characteristics:** Engineering background; focused on performance and reliability metrics rather than user experience.

4. KMD Technology Solutions Software Team (Secondary Users)

- **Role Responsibilities:** Oversee integration between the AIBI vision module and KMD's existing workflow management platform; validate data exchange and API compatibility.
- **User Priority:** Secondary User.
- **Subject Matter Experience:** Master - expert knowledge of KMD systems and factory workflow data structures.
- **Technological Experience:** Master - proficient in API development and system architecture.
- **Other Characteristics:** Focused on data consistency and long-term interoperability requirements.

5. AIBI Solutions Management and Client Representatives (Secondary Users)

- **Role Responsibilities:** Review reports, set project goals, and evaluate system performance for business impact and compliance with security requirements.
- **User Priority:** Secondary User - stakeholders for project success and regulatory alignment.
- **Subject Matter Experience:** Journeyman - familiar with manufacturing operations and AI deployment constraints.
- **Technological Experience:** Journeyman - comfortable with analytics dashboards and reporting tools.
- **Other Characteristics:** Interested in ROI, reliability, and compliance.

4.1.3. Functional Requirements

4.1.3.1. Project Scope (Use Case Diagram)

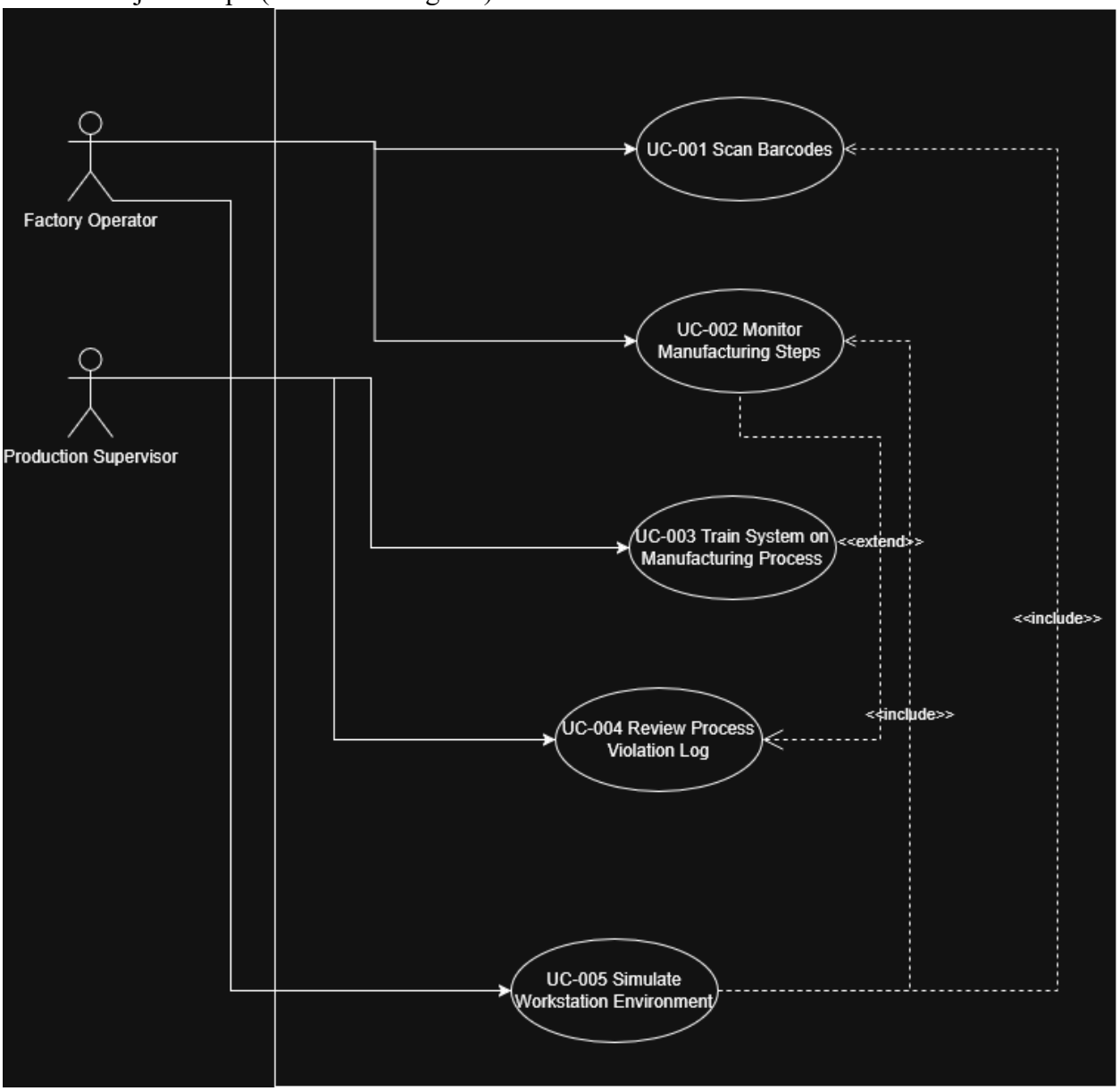

Figure 4.1.3.1.1 Use Case Diagram

The Use Case diagram in Figure 4.1.3.1.1 consists of 2 actors: Factory Operator and Production Supervisor. Factory Operator is the primary actor for UC-001 and UC-002. UC-002 includes the functionality of UC-003, which is training the system. This is required for the model to work. UC-003 and UC-004 will have the production supervisor as the main actor. The production supervisor is responsible for training the system so it can monitor the steps properly. The supervisor also has access to the Violation Log which takes the functionality of UC-002 to get the errors.

4.1.3.2. User Scenarios

UC-001: Scan Part Barcodes

- A factory operator places a paper containing barcodes within the camera's field of view. The system automatically detects and decodes all visible barcodes, maps them to configured data fields, and outputs structured JSON data to the downstream system. If barcodes fail to decode, the system alerts the operator.

UC-002: Monitor Manufacturing Step

- A factory operator performs manufacturing steps while the system continuously monitors the video feed. The system detects when each step begins and ends, validates the correct sequence, and logs completion with timestamps. If a step is performed out of sequence or skipped, the system displays alerts and prevents progression until corrected.

UC-003: Train System on Manufacturing Process

- A production supervisor records a reference video of an experienced operator performing a new manufacturing process. The supervisor marks the start and end of each step, labels them, and adds notes. The system saves the training session with metadata for use in future process verification.

UC-004: Review Process Violation Log

- A production supervisor opens the violation log interface and filters by date, workstation, or violation type. The supervisor selects individual violations to view detailed information including timestamps, video footage, and operator IDs. After review, the supervisor marks violations as reviewed and exports reports for management analysis.

UC-005: Simulate Workstation Environment

- A developer loads recorded camera footage into the simulation environment and injects synthetic barcode data to test system functionality. The system processes the simulated inputs, generates outputs, and displays verification results. The developer validates system behavior before production deployment.

4.1.3.3. User Functional Requirements

UFR1. Fixed cameras at each workstation must detect, decode, and process multiple barcodes from objects presented within the camera’s field of view

UFR2. Provide configurable mapping system to assign barcode positions to specific data fields

UFR3. Identify and track QR codes or barcodes on large parts rather than sheets of paper

UFR4. Aggregate all decoded values into structured JSON payload for downstream system integration

UFR5. Monitor and validate that manufacturing steps are performed in correct sequence at each workstation

UFR6. Provide immediate visual feedback and alerts for process status and violations on terminals at workstations

UFR7. Log all errors caught by the system for the users to review

UFR8. Provide intuitive UI for non-technical users to train the system on correct manufacturing processes

UFR9. Seamless integration with existing KMD Technology Solutions workflow management system

UFR10. Implement intermediate data storage to decouple computer vision processing from KMD data consumption

UFR11. Provide simulated workstation environment for development and validation before production deployment

UFR12. Provide tools for capturing, organizing, and managing training datasets for model development

4.1.4. Non-functional Requirements

4.1.4.1. Product: Usability Requirements

PUUR1. System must be extremely easy to use for manufacturing workers with minimal training, needs to be easier than what is currently being done

PUUR2. Minimize physical interaction requirements during normal operation

PUUR3. Training and configuration interfaces must be accessible to non-technical staff

PUUR4. Design system to accommodate workers with varying technical skills and possible disabilities

4.1.4.2. Product: Performance Requirements

PPUR1. Process video feeds and provide feedback with minimal latency to prevent manufacturing errors

PPUR2. Maintain high accuracy in both barcode detection and process monitoring for aerospace quality standards

PPUR3. Function effectively with lower-quality, cost-effective camera equipment

4.1.4.3. Product: Dependability/Reliability/Security

PDUR1. All processing and data storage must remain on-premises with no cloud dependencies or external services

PDUR2. Reliable operation in industrial factory conditions with minimal downtime

4.1.4.4. Organizational: Development Requirements

ODUR1. Utilize open-source libraries and frameworks to minimize licensing costs

4.1.4.5. Organizational: Operational Requirements

OOUR1. Minimize hardware and deployment costs while meeting performance requirements

OOUR2. Support flexible deployment configurations for different workstation setups

OOUR3. Support gradual implementation with minimal disruption to existing operations

4.1.4.6. External: Interoperability Requirements

EIUR1. Maintain compatibility with existing KMD Technology Solutions software interfaces

EIUR2. Publish comprehensive JSON schema for scan results and system outputs

EIUR3. Support integration with existing manufacturing systems and processes

4.2. System Requirements

4.2.1. Functional Requirements

4.2.1.1. System Functional Requirements

SFR1. Fixed-station cameras shall detect, decode, and output all barcodes or QR codes visible in a single frame

SFR2. Provide per-workstation configurable mapping to assign barcode spatial position to named data fields

SFR3. Recognize and extract etched/carved alphanumeric identifiers on large parts

SFR4. Aggregate decoded and recognized values into a structured JSON document per scan event

SFR5. Support real-time delivery via local REST API, local DB persistence, and keyboard-emulation mode for KMD integration

SFR6. Detect and report start/completion of defined manufacturing steps and validate configured sequence; flag skipped or out of sequence steps

SFR7. Provide per workstation UI overlay with live video, step progress indicators, visual status, and optional audio alerts for violations

SFR8. Provide an in-site capture tool for non-technical users to record reference video, mark step start/end, label steps, and save training sessions with metadata

SFR9. Models must tolerate natural worker variations and provide configurable sensitivity per step to reduce nuisance alerts

SFR10. The system shall log every detected incorrect, skipped, or out of sequence manufacturing step and save for review

SFR11. Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable

SFR12. Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields

SFR13. Provide a simulated workstation environment to replay camera feeds, inject synthetic barcodes/etched IDs, and verify API/keyboard-emulation outputs before production deployment

SFR14. Provide tools and storage to capture, tag, version, and export labeled datasets for model training, including metadata

SFR15. Fixed-station cameras shall decode all barcodes or QR codes visible in a single frame

SFR16. Fixed-station cameras shall output all barcodes or QR codes visible in a single frame

4.2.1.2. Data Requirements

The system depends on a combination of real-time camera data, barcode information, and process-tracking records to support its monitoring and verification functions. Each data source must be handled securely and consistently to ensure accuracy and reliability in a manufacturing environment.

The main input data comes from live video streams captured by cameras at each workstation. These frames are processed locally on edge devices to identify barcodes, track parts, and verify worker actions. Each frame includes a timestamp and workstation ID to maintain synchronization and traceability across multiple cameras.

All data is stored securely on the local network and managed according to the manufacturer's privacy and security policies. Sensitive information, such as worker identifiers or connection credentials, is protected using encrypted .env files and role-based access control. These data requirements ensure that the system maintains accurate and traceable records of all part and process events, supporting efficient monitoring, auditing, and future scalability.

4.2.2. Non-functional Requirements

4.2.2.1. Product: Usability Requirements

PUSR1. The UI shall be intuitive and require no more than 30 minutes of training for first time users

PUSR2. The system shall provide clear on-screen guidance and tooltips for all major functions

PUSR3. The UI shall minimize the amount of effort required to complete required tasks compared to the existing process

PUSR4. The system shall support touch-free operation through automatic detection and feedback mechanisms where possible

PUSR5. The system shall automate common workflow steps to reduce manual input from operators

PUSR6. The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon

PUSR7. The system shall include a guided setup and tutorial modes for initial configuration

PUSR8. The system shall allow non-administrative users to modify basic settings without needing programming knowledge.

PUSR9. The interface shall include support for large text and high-contrast display options

PUSR10. The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff

PUSR11. The system shall comply with accessibility standards

PUSR12. The system shall support both mouse and keyboard input/navigation for inclusivity

4.2.2.2. Product: Performance Requirements

PPSR1. The system shall process video input and deliver analysis feedback within 200 milliseconds

PPSR2. The system shall prioritize real-time operations over background tasks to minimize latency

PPSR3. The system shall achieve a barcode detection accuracy of at most one error per 100 scans.

PPSR4. The system shall detect and report process deviations with an error margin of less than 1%

PPSR5. The system shall maintain required accuracy levels using cameras with resolutions as low as 720p

PPSR6. The system shall automatically adjust detection parameters for varying camera quality and lighting conditions

4.2.2.3. Product: Dependability/Reliability/Security

PDSR1. The system shall store all data on local servers within the facility

PDSR2. The system shall perform all processing operations using on-premises hardware without cloud reliance

PDSR3. The system shall maintain a minimum uptime of 99% during operational hours

PDSR4. The system shall be tested for stable operation under factory environmental conditions

4.2.2.4. Organizational: Development Requirements

ODSR1. The system shall use open-source libraries or frameworks unless a commercial dependency is explicitly approved

ODSR2. All open-source components shall be documented, including their licenses and version numbers

4.2.2.5. Organizational: Operational Requirements

OOSR1. The system shall maintain performance benchmarks on standard hardware

OOSR2. The installation process shall include customizable deployment options

OOSR3. The system shall support phased deployment alongside existing systems

OOSR4. The system shall maintain backward compatibility with legacy data formats

4.2.2.6. External: Interoperability Requirements

EISR1. The system shall integrate with all existing KMD Technology Solutions APIs and communication protocols

EISR2. The system shall maintain consistent data exchange formats used by KMD

EISR3. The system shall define and publish a complete JSON schema for all output data

EISR4. The JSON schema shall be version-controlled and included in system documentation

EISR5. The system shall support data exchange with existing Manufacturing Execution Systems and ERP tools

EISR6. The system shall be configurable to align with existing factory workflows and data pipelines

4.3. Requirements Trace Table

The trace table, at the bottom of Appedix R, has all the mappings from user to system requirements. The table is used as a reference to see what system requirements were engineered to system requirements.

5. Exploratory Studies

5.1. Relevant Techniques

The exploratory phase focused on identifying computer-vision techniques that could operate reliably in real manufacturing environments without constant retraining or cloud support. Because each facility may use different layouts, materials, and workflows, the system needed to be flexible, lightweight, and fully on-premises.

For part and material tracking, traditional vision methods proved the most effective. Barcode and QR decoding libraries such as pyzbar [1] and ZXing [2] were evaluated as dependable solutions for linking visual data to unique part identifiers. These techniques provide precise and verifiable tracking of materials as they move through each stage of production, forming the backbone of the system's traceability layer.

For worker-step verification, a more adaptive and semantic approach was needed. The project explored OpenCLIP [3] and MobileCLIP [4], which generate multimodal embeddings capable of connecting visual input to textual task descriptions. By comparing live camera frames against step definitions or reference images encoded in the same embedding space, the system can determine whether the correct procedure is being followed without requiring model retraining for each new process.

This hybrid strategy, symbolic tracking for materials, semantic understanding for actions, allows the system to remain both accurate and flexible. Edge optimization and model quantization were also studied to ensure that OpenCLIP [3] can operate efficiently on local GPU hardware, maintaining real-time performance while keeping all data processing securely on-site.

5.2. Relevant Packages/Products

Several open-source frameworks and libraries were examined during the exploratory phase. OpenCV [5] remained the foundation for video capture, preprocessing, and visualization. OpenCLIP [3] was integrated through the PyTorch [6] ecosystem for embedding generation and similarity matching. Though we cannot use the MobileCLIP [4] model itself as it falls under a limit research license. FastAPI [7] was reviewed for real-time data exchange between the vision modules, the LLM-based reasoning system, and the operator dashboard. Data management and analysis relied on NumPy [8], Pandas [9], and SQL databases to record event logs and validation results. Package management tools such as uv [10] and conda [11] were evaluated for reproducible, isolated deployments suitable for regulated installations.

5.3. Broader Impacts

Using OpenCLIP [3] expands what's possible for on-premises AI in manufacturing. These models allow semantic understanding of human actions and part interactions without requiring cloud-scale datasets or manual labeling, opening the door to more adaptive and intelligent factory systems.

In the long term, this technology can improve safety by recognizing incorrect procedures in real time, support training by providing immediate visual feedback, and streamline audits by automatically logging validated process steps. More broadly, it demonstrates how advanced AI can be localized, offering powerful visual reasoning while preserving data ownership, privacy, and compliance.

6. System Design

6.1. Architectural Design

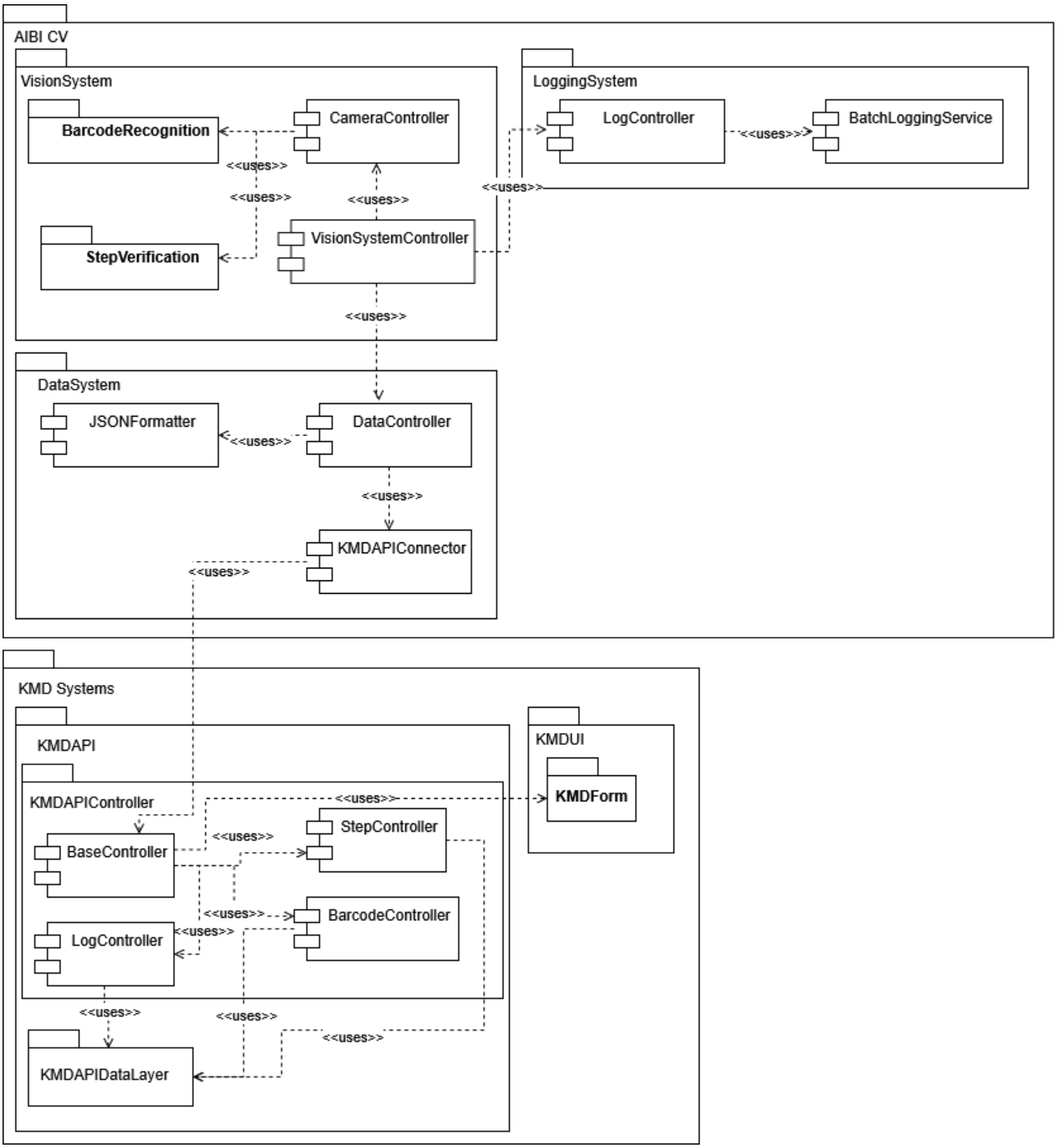

Figure 6.1 UML Package Diagram

The UML package diagram in Figure 6.1 provides a high-level overview of the system structure. It consists of two main packages: AIBI CV and KMD Systems. The AIBI CV package shows the system responsible for barcode scanning and step verification. Within the VisionSystem

package, the VisionSystemController orchestrates these processes via the CameraController, utilizing the BarcodeRecognition and StepVerification packages. It also manages batching logs generated from these operations. The VisionSystemController then leverages the DataController and JSONFormatter to format the data before sending it to the KMDAPI through the KMDAPIConnector.

The KMD Systems package contains existing KMD components that the system will integrate with. Requests from the KMDAPIConnector are handled by the BaseController in the KMDAPIConnector package. The BaseController routes requests to the appropriate controller for Step Verification, Barcode Recognition, or Logging. These controllers store data using KMD's existing KMDAPIDataLayer and also forward relevant information to the KMDUI through the BaseController.

7. System Implementation

7.1. Programming Languages & Tools

The system is developed using Python and modern open-source tools optimized for on-premise deployment, reproducibility, and performance. Python was chosen for its rich ecosystem of computer vision, machine learning, and systems integration libraries, as well as its readability and flexibility for both research and production environments.

Primary Language

- **Python 3.10+**: Core language for all image processing, barcode decoding, and logic components. Python's mature libraries support rapid development and integration with existing AI toolkits.

Supporting Tools and Libraries

- **OpenCV [5]**: Provides video-frame capture, image preprocessing (contrast enhancement, thresholding, edge detection), and visualization.
- **Pyzbar [1] / ZXing [2]**: Enables decoding of standard barcodes and QR codes from image frames.
- **open_clip [3]:** Constructs embeddings of the current image, which can be used in an algorithm for worker-step verification system
- **NumPy [8] / Pandas [9]**: Used for numerical operations, data analysis, and structured logging of scan events.
- **FastAPI [7]**: Lightweight web framework for creating administrative dashboards, REST endpoints, and configuration interfaces.
- **SQL Server**: Database back end for storing scan results, process validation data, and system configuration. Existing systems already use SQL Server, and are currently running on an on-premise server.
- **Uv [10]**: Used for environment and dependency management. Chosen for its speed, reproducibility, and ability to handle isolated Python environments more efficiently than pip or Poetry.
- **Git + GitHub**: Source control and collaboration platform for version tracking, documentation, and code review.
- **Ubuntu Server 22.04 LTS**: Target deployment environment, ensuring compatibility, long-term support, and security for on-premise installation.
- **VS Code**: Primary IDE for development and debugging.

Optional / Future Tools

- **Dynamsoft Barcode SDK**: Considered for enhanced barcode decoding robustness in poor lighting or complex conditions.

- **Docker (TBD)**: May be adopted for containerized deployment and easier replication across multiple workstations.

This toolchain emphasizes efficiency, security, and flexibility. Using Python and OpenCV [5] enable modular, camera-driven computer vision workflows that align with industry standards.

7.2. Coding Conventions

To maintain a consistent, readable, and maintainable codebase across the team, the project follows standardized Python development practices. These conventions support collaboration, ease of debugging, and future extensibility.

The development environment consists of Python 3.10+ on Ubuntu 22.04 LTS. We manage the dependencies using the uv [10] library, which uses pyproject.toml and uv.lock to ensure reproducible builds.

The use of inline comments to explain what the code does and why the decisions were made will keep the code easy to read and continue further development on. Proper naming conventions will be utilized in files, classes, methods, and variables to promote readability. Each subsystem will include a short README.md to explain the use of said subsystem. Using Python's built-in logging, we will be able to produce a verbose output that will let developers know exactly where the code failed.

7.3. Code Version Control

All project code is managed using Git and hosted on GitHub, providing a centralized platform for version tracking, collaboration, and peer review. The team follows a structured branching workflow, with the main branch reserved for stable, production-ready releases and a dev branch used for active development and integration testing. New features or fixes are developed in dedicated feature branches (e.g., feature/<name> or fix/<name>), ensuring that changes remain isolated until verified. Each commit will include a detailed message of what was changed. All pull requests must be reviewed by at least one team member and successfully pass automated testing before merging. Sensitive information, including API keys and credentials, is secured through .env files and excluded from the repository via .gitignore. Access to the repository is role-based, with maintainers responsible for approving merges into the main branch to preserve stability and code integrity.

9. Challenges & Open Issues

9.1. Challenges Faced in Requirements Engineering

Defining clear requirements for this project has been an ongoing challenge due to its evolving scope and layered communication structure. Unlike a traditional software project with fixed specifications, this initiative is still in the developmental phase within the manufacturer's broader modernization effort. As a result, many system expectations continue to shift as the manufacturer refines its own internal processes and priorities.

Another difficulty stems from the project's two-in-one nature. AIBI Solutions is developing the AI component while KMD Technology Solutions oversees the larger software integration. This means that decisions are often split between technical design for AIBI's on-premise AI system and operational requirements defined by KMD's project management tools. Balancing both sides has required ongoing clarification to ensure that features remain aligned with the manufacturer's real-world workflow.

Communication flow also adds complexity. Because the development is being conducted for a manufacturer through a software company, information moves through several layers before reaching the engineering team. Questions and feedback can take time to travel back and forth, which slows iteration and can cause temporary gaps in understanding.

Additionally, from the manufacturer's perspective, the AI system is currently viewed as a lower priority compared to its core production and infrastructure initiatives. This limits the amount of dedicated feedback and testing time available, making it harder to gather detailed operational requirements early on.

Many questions at this stage remain speculative, there are still unknowns about how the system will integrate with existing machinery, what data streams will be available, and what the long-term performance expectations will look like. Because of that uncertainty, much of the early requirements work has focused on flexibility and modularity rather than strict definitions.

Finally, testing in a production-like environment presents logistical challenges. Access to the actual manufacturing floor is limited, and replicating realistic lighting, noise, and workflow conditions off-site is difficult. This means that validating the system's accuracy and reliability under true operating conditions will likely occur later in the development cycle, increasing the importance of careful simulation and staged deployment.

Overall, these challenges have required an adaptive, iterative approach to requirements engineering, focusing on communication, flexibility, and traceability while allowing the technical design to evolve alongside the manufacturer's own readiness.

10. System Manuals

10.1. Instructions for System Development

System development for this project follows a structured process designed to maintain consistency, reliability, and reproducibility across all team members and deployment environments. All development is conducted using Python 3.10+ on Ubuntu Server 22.04 LTS, ensuring compatibility with the on-premise infrastructure already in place at the manufacturer. Dependency management is handled through uv [10], which provides deterministic builds which enable developers to quickly synchronize environments across different workstations. Development and debugging are performed using VS Code, chosen for its integration with Git, Python, and FastAPI-based tooling.

All source code is tracked using Git and hosted on GitHub, following the branching workflow established in Section 7.3. The main branch is reserved for stable, production-ready code, while all active work occurs in the dev branch. All merges require successful test execution and peer review. Commit messages follow a consistent format describing the scope of each change. Sensitive credentials, including database connection strings and API keys, are never committed to version control and are securely managed through .env files that are ignored by Git.

The system is organized into modular subsystems to ensure scalability and maintainability. The barcode recognition subsystem uses OpenCV [5] for image preprocessing and Pyzbar [1] / ZXing [2] for decoding one-dimensional and two-dimensional barcodes, while the worker-step verification subsystem utilizes OpenCV and open_clip [3] to extract image embeddings and verify human task completion through step-specific inference. Both subsystems rely on NumPy [8] and Pandas [9] for numerical processing and structured data logging. Logging throughout the system is handled using Python's built-in logging library, providing detailed runtime information to assist in debugging and error tracing. Each subsystem includes a README.md file describing its purpose, structure, and key functions, ensuring that future developers can quickly understand and extend the implementation.

All scan results, worker verification data, and configuration settings are stored in an on-premises SQL Server database. Connections are established through environment-defined credentials and utilize parameterized queries to maintain data integrity and security. Testing is an integral part of the development workflow: unit tests are written with pytest while integration testing is conducted using sample camera feeds and controlled production simulations. Benchmarking is used to evaluate performance metrics such as frame rate, recognition accuracy, and latency under realistic load conditions.

Deployment targets include both edge workstations and a central application server, each running isolated uv environments for reproducibility. Logs and configuration files are maintained in standardized directories on the host system. All changes to the system are documented through inline code comments, updated README.md files, and version entries in CHANGELOG.md.

By adhering to these development instructions, the team ensures that the AIBI computer-vision system remains modular, secure, and reproducible. The consistent use of Python, uv, and GitHub

provides a reliable foundation for collaboration, testing, and long-term maintainability, while the project's standardized structure supports ongoing refinement and expansion as new features and subsystems are introduced.

12. References

[1] L. Hudson, “Naturalhistorymuseum/Pyzbar: Read One-dimensional barcodes and QR codes from Python 2 and 3.,” GitHub, https://github.com/NaturalHistoryMuseum/pyzbar (accessed Oct. 13, 2025).

[2] “Zxing/Zxing: Zxing (‘Zebra crossing’) Barcode Scanning Library for Java, Android,” GitHub, https://github.com/zxing/zxing (accessed Oct. 13, 2025).

[3] G. Ilharco et al., OpenCLIP. Zenodo, 2021. doi: 10.5281/zenodo.5143773 (accessed Oct. 13, 2025).

[4] F. Faghri et al., “MobileCLIP2: Improving Multi-Modal Reinforced Training,” Transactions on Machine Learning Research, 2025, [Online]. Available: https://openreview.net/forum?id=WeF9zolng8 (accessed Oct. 13, 2025).

[5] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000 (accessed Oct. 13, 2025).

[6] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” 2019. [Online]. Available: https://arxiv.org/abs/1912.01703 (accessed Oct. 13, 2025).

[7] S. Ramírez, FastAPI. [Online]. Available: https://github.com/FastAPI/FastAPI (accessed Oct. 13, 2025).

[8] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2 (accessed Oct. 13, 2025).

[9] The pandas development team, pandas-dev/pandas: pandas. doi: 10.5281/zenodo.3509134 (accessed Oct. 13, 2025).

[10] C. Marsh et al., “uv,” UV, https://docs.astral.sh/uv/#the-pip-interface (accessed Oct. 13, 2025).

[11] conda contributors, conda: A system-level, binary package and environment manager running on all major operating systems and platforms. [Online]. Available: https://github.com/conda/conda (accessed Oct. 13, 2025).

13. Team Members' Log Sheets

13.1 Nathan DiGilio

Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
10/02/2025	Mentor/advisor meetings, group discussion	3 hours
10/8/2025	Problem Statement section	4 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/9/2025	Exploratory Studies	5 hours
10/10/2025	System Implementation section	4 hours
10/11/2025	Challenges & Open Issues	4 hours
10/13/2025	Report Formatting	3 hours
		Total: 29 hours

13.2 Jacob Dzikowski

Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
10/02/2025	Mentor/advisor meetings, group discussion. Initial document creation (outline, title page, etc.)	5 hours
10/07/2025	Initial abstract draft, added some requirements	2 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/11/2025	Use cases, table generation, general improvements	5 hours
10/12/2025	Requirements table generation, general improvements	4 hours
10/13/2025	General improvements, formatting, etc.	10 hours
		Total: 32 hours

13.3 Daniel Pora

Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
9/30/2025	Created user nonfunctional and functional requirements from 9/19 meeting	2.5 hours
10/02/2025	Wrote questions for Sponsor, from advisor meeting and group discussion	2.5 hours
10/02/2025	Mentor/advisor meetings, group discussion	3 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/09/2025	Created user nonfunctional and functional requirements from 10/02 meeting	2.5 hours
10/09/2025	Combined user nonfunctional and functional requirements into single document for system conversion	1.5 hours
10/12/2025	Worked on report	4 hours
10/13/2025	Worked on report	8 hours
		Total: 30 hours

13.4 Collin Miller

Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
10/02/2025	Mentor/advisor meetings, group discussion	3 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/10/2025	Converted User to System Requirements; Made some User Requirement Cards	4 hours
10/11/2025	Added System Requirements to the document and reviewed them	4 hour
10/11/2025-10/13/2025	Made all System Requirement cards; Made the trace table; Worked in general on report; Checked over all requirement	16 hours

	cards to make sure the traces were correct; Worked on use case diagram	
		Total: 33 hours

Appendix U

1. Use Case Tables

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-001
Use Case Name:	Scan Part Barcodes
User Goal:	Automatically detect and decode multiple barcodes at a workstation
Scope:	Barcode Recognition Module
Level:	Primary Task
Primary Actor:	Factory Operator
Precondition:	Camera system is operational, and workstation is configured
Minimal Guarantee:	System logs scan attempt
Success Guarantee:	All barcodes decoded and data formatted into JSON output
Trigger:	Paper with barcodes is placed within camera field of view
Success Scenario:	**Step Action**
	1. Factory Operator places part with visible barcodes within camera field of view
	2. System detects object presence in frame
	3. System captures video frame
	4. System detects all visible barcodes/QR codes in frame
	5. System decodes each barcode and extracts data
	6. System maps decoded values to configured data fields based on spatial position
	7. System aggregates all decoded values into structured JSON payload
	8. System outputs JSON to configured destination (API, database, or keyboard emulation)
	9. System displays success feedback on workstation terminal
Extensions:	**Branching Scenarios**
4A	**Condition:** No barcodes detected
	Step Actions
	1. System logs detection failure
	2. System displays warning message to operator
	3. System waits for operator to reposition paper with barcodes

	4. Continue from main step 2
5A	**Condition:** Barcode decode failure
	Step Actions
	1. System logs which barcode(s) failed to decode
	2. System displays error message indicating failure
	3. System prompts operator to clean or reposition paper with barcodes, or enter information manually
	4. Continue from main step 2
8A	**Condition:** Downstream system unavailable
	Step Actions
	1. System stores JSON payload locally with timestamp
	2. System displays warning to operator
	3. System queues data for later synchronization
	4. Continue from main step 9

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-002
Use Case Name:	Monitor Manufacturing Steps
User Goal:	Verify that manufacturing steps are performed in correct sequence
Scope:	Step Verification Module
Level:	Primary Task
Primary Actor:	Factory Operator
Precondition:	Process steps are configured and camera system is operational
Minimal Guarantee:	System logs monitoring attempt
Success Guarantee:	Step verified as complete and logged correctly
Trigger:	Operator begins manufacturing step within view of camera
Success Scenario:	**Step Action**
	1. Factory Operator begins manufacturing step
	2. System continuously analyzes video feed

	3. System detects step start based on configured visual indicators
	4. System updates UI overlay to show step in progress
	5. System monitors for step completion indicators
	6. System detects step completion
	7. System validates step occurred in correct sequence
	8. System logs step completion with timestamp and metadata
	9. System updates UI to show step complete
	10. System enables next step in sequence
Extensions:	**Branching Scenarios**
7A	**Condition:** Step performed out of sequence
	Step Actions
	1. System detects sequence violation 2. System displays visual alert on workstation terminal 3. System plays optional audio alert 4. System logs violation with details 5. System prevents progression to next step 6. Production Supervisor reviews violation log 7. Use case ends
7B	**Condition:** Step skipped
	Step Actions
	1. System detects operator attempting to skip to later step 2. System displays warning message 3. System logs skipped step violation 4. System prevents progression until skipped step completed 5. Continue from main step 1
6A	**Condition:** Step not completed within expected time
	Step Actions
	1. System detects timeout 2. System displays warning to operator 3. System logs delay 4. System continues monitoring 5. Continue from main step 5

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-003
Use Case Name:	Train System on Manufacturing Process

User Goal:	Record and label reference video of correct manufacturing process
Scope:	Training Data Capture Module
Level:	Primary Task
Primary Actor:	Production Supervisor
Precondition:	User has access to training interface and camera system is operational
Minimal Guarantee:	Training sessions are saved with basic metadata
Success Guarantee:	Complete training dataset captured with all step labels and metadata
Trigger:	Supervisor selects "New Training Session" from system interface
Success Scenario:	**Step Action**
	1. Production Supervisor launches training capture tool
	2. System displays live video feed and training controls
	3. Supervisor enters process name and description
	4. Supervisor clicks "Start Recording"
	5. System begins recording video feed
	6. Operator performs first manufacturing step
	7. Supervisor clicks "Mark Step Start"
	8. System records timestamp and frame number
	9. Operator completes first step
	10. Supervisor clicks "Mark Step End"
	11. System prompts for step label/description
	12. Supervisor enters step label
	13. Steps 6-12 repeat for each manufacturing step
	14. Supervisor clicks "Complete Training"
	15. System saves training session with all metadata
	16. System displays confirmation message
Extensions:	**Branching Scenarios**
7A	**Condition:** Supervisor marks incorrect start point
	Step Actions

	1. Supervisor clicks "Undo Last Mark" 2. System removes most recent marker 3. Continue from main step 7
12A	**Condition:** Supervisor wants to add notes to step
	Step Actions
	1. System logs which barcode(s) failed to decode1. System displays additional notes field 2. Supervisor enters detailed notes 3. System saves notes with step metadata 4. Continue from main step 13
15A	**Condition:** Storage unavailable
	Step Actions
	1. System displays error message 2. System saves training data to temporary location 3. System notifies supervisor of temporary save 4. Use case ends

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-004
Use Case Name:	Review Process Violation Log
User Goal:	Review and analyze logged process violations
Scope:	Logging and Reporting Module
Level:	Primary Task
Primary Actor:	Production Supervisor
Precondition:	Supervisor has access to logging interface
Minimal Guarantee:	Log data remains unchanged
Success Guarantee:	Supervisor reviews violations and takes appropriate action
Trigger:	Supervisor selects "View Violation Log" from system menu
Success Scenario:	**Step Action**
	1. Production Supervisor opens violation log interface
	2. System displays list of all logged violations
	3. Supervisor filters log by date range, workstation, or violation type
	4. System updates display to show filtered results
	5. Supervisor selects specific violation entry

	6. System displays detailed violation information (timestamp, type, step, operator)
	7. System displays any associated video footage or screenshots
	8. Supervisor reviews details
	9. Supervisor marks violation as "Reviewed"
	10. System updates violation status
	11. Supervisor repeats steps 5-10 for additional violations
	12. Supervisor exports log data for reporting
	13. System generates report in requested format
Extensions:	**Branching Scenarios**
6A	**Condition:** No details available for violation
	Step Actions
	1. System displays "Details unavailable" message 2. System shows basic log entry only 3. Continue from main step 8
13A	**Condition:** Export fails
	Step Actions
	1. System displays error message 2. System logs export failure 3. Supervisor retries or contacts support 4. Use case ends

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-005
Use Case Name:	Simulate Workstation Environment
User Goal:	Test system functionality before production deployment
Scope:	Simulation and Testing Module
Level:	Primary Task
Primary Actor:	KMD Technology Solutions Software Team
Precondition:	Simulation environment is installed and configured
Minimal Guarantee:	Simulation data logged
Success Guarantee:	System tested and validated in simulated environment

Trigger:	Developer selects "Start Simulation" from testing interface
Success Scenario:	**Step Action**
	1. Developer launches simulation environment
	2. System displays simulation controls
	3. Developer loads recorded camera feed
	4. System displays video playback controls
	5. Developer injects synthetic barcode data
	6. System processes simulated input
	7. System generates JSON output
	8. Developer verifies JSON format and content
	9. Developer tests API/keyboard-emulation outputs
	10. System displays output verification results
	11. Developer reviews results and logs
Extensions:	**Branching Scenarios**
6A	**Condition:** Processing error in simulation
	Step Actions
	1. System logs error details 2. System displays error message to developer 3. Developer reviews error logs 4. Developer adjusts simulation parameters 5. Continue from main step 5
8A	**Condition:** JSON validation fails
	Step Actions
	1. System highlights validation errors 2. Developer reviews schema mismatches 3. Developer corrects configuration 4. Continue from main step 6
10A	**Condition:** Output verification fails
	Step Actions
	1. System displays specific failure points 2. Developer reviews system configuration 3. Developer makes necessary adjustments 4. Continue from main step 9

Appendix R

1. User Functional Requirements:

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR1		Type	Functional	Non-Functional
Creation	October 9, 2025, 3:51pm		User	✓	
Modification	October 9, 2025, 3:51pm		System		
Description	Fixed camera at each workstation must detect, decode, and process multiple barcodes from objects presented within the camera's field of view				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR1, SFR4, PPSR2, PPSR3, SFR15, SFR16				
Justify why UFR1 can be completely covered by SFR1, SFR4, PPSR1, PPSR2, PPSR3	The barcode will scan a barcode or QR code and then decode those values into a structured JSON format prioritizing it happening in real time with an accuracy of at most 1 error per 100 scans				
Traceability	Use cases	UC-001, UC-005			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR2		Type	Functional	Non-Functional
Creation	October 9, 2025, 3:54pm		User	✓	
Modification			System		
Description	Provide configurable mapping system to assign barcode positions to specific data fields				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR2				
Justify why UFR2 can be completely covered by SFR2	This will allow a configurable mapping for barcodes in certain positions in each workstation				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR3		Type	Functional	Non-Functional
Creation	October 12, 2025, 8:38pm		User	✓	
Modification			System		
Description	Identify and track QR codes or barcodes on large parts rather than sheets of paper				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR3, PPSR5, PPSR6				
Justify why UFR3 can be completely covered by SFR3, PPSR5, PPSR6	The camera will recognize any barcode or etching on any part with great accuracy levels and will adjust with varying camera qualities and lighting conditions				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR4		Type	Functional	Non-Functional
Creation	October 12, 2025, 9:10pm		User	✓	
Modification			System		
Description	Aggregate all decoded values into structured JSON payload for downstream system integration				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR4, SFR12, EISR3, EISR4				
Justify why UFR4 can be completely covered by SFR4, SFR12, EISR3, EISR4	The camera will take the scanned data and put it into a structed JSON and be published. It will be version-controlled and included in system documentation				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR5		Type	Functional	Non-Functional
Creation	10/12/2025, 9:11pm		User	✓	
Modification			System		
Description	Monitor and validate that manufacturing steps are performed in correct sequence at each workstation				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR6, SFR10, PPSR4,				
Justify why UFR5 can be completely covered by SFR6, SFR10, PPSR4	SFR6 ensures the system detects and validates each manufacturing step in sequence, while SFR10 logs any deviations. PPSR4 guarantees that the system flags errors with less than 1% margin, fulfilling the need for accurate step monitoring and traceability.				
Traceability	Use cases	UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR6		Type	Functional	Non-Functional
Creation	10/12/2025, 9:12pm		User	✓	
Modification			System		
Description	Provide immediate visual feedback and alerts for process status and violations on terminals at workstations				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR7, PPSR1				
Justify why UFR6 can be completely covered by SFR7, PPSR1	SFR7 provides live UI overlays and alerts, giving operators immediate feedback. PPSR1 ensures this feedback is delivered within 200ms, meeting the need for real-time responsiveness and visual clarity at workstations				
Traceability	Use cases	UC-002, UC-004			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	UFR7	Type	Functional	Non-Functional
Creation	10/12/2025, 9:13pm	User	✓	

Modification				System		
Description	Log all errors caught by the system for the users to review				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into	SFR10, SFR11, PDSR1, PDSR2, PDSR3				
Justify why 7 can be completely covered by SFR10, SFR11, PDSR1, PDSR2, PDSR3	SFR10 logs all process violations, while SFR11 ensures local storage and replay capabilities. PDSR1 and PDSR2 enforce on-premises data handling, and PDSR3 guarantees system uptime, collectively supporting reliable error tracking and review.				
Traceability	Use cases	UC-004			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR8			Type	Functional	Non-Functional
Creation	10/12/2025, 9:14pm			User	✓	
Modification				System		
Description	Provide intuitive UI for non-technical users to train the system on correct manufacturing processes				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR8, SFR9, PUSR1, PUSR2, PUSR3, PUSR4, PUSR5, PUSR6, PUSR7, PUSR8, PUSR9, PUSR10				
Justify why UFR8 can be completely covered by SFR8, SFR9, PUSR1-PUSR10	SFR8 enables non-technical users to record and label training videos. SFR9 ensures the system adapts to natural worker variations. The PUSR requirements collectively ensure the UI is intuitive, accessible, and easy to configure, fulfilling the need for a user-friendly training interface.				
Traceability	Use cases	UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	UFR9	Type	Functional	Non-Functional
Creation	10/12/2025, 9:14pm	User	✓	
Modification		System		
Description	Seamless integration with existing KMD Technology Solutions workflow management system			

Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into		SFR5, SFR12			
Justify why UFR9 can be completely covered by SFR5, SFR12		SFR5 supports integration via REST API and keyboard emulation, while SFR12 ensures consistent JSON schema documentation. Together, they enable seamless interoperability with KMD's workflow system.			
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR10		Type	Functional	Non-Functional
Creation	10/12/2025, 9:16pm		User	✓	
Modification			System		
Description	Implement intermediate data storage to decouple computer vision processing from KMD data consumption				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into		SFR11, PDSR1, PDSR2			
Justify why 10 can be completely covered by SFR11, PDSR1, PDSR2		SFR11 provides intermediate local storage and replay capabilities. PDSR1 and PDSR2 ensure all data remains on-premises, fulfilling the need to decouple vision processing from KMD's data consumption securely.			
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR11		Type	Functional	Non-Functional
Creation	10/12/2025, 9:16pm		User	✓	
Modification			System		
Description	Provide simulated workstation environment for development and validation before production deployment				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		SFR13, PDSR4			

Justify why UFR11 can be completely covered by SFR13, PDSR4		SFR13 enables simulation of workstation environments for testing. PDSR4 ensures the system is validated under real factory conditions, supporting safe and reliable pre-deployment testing.
Traceability	Use cases	UC-005
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR12			Type	Functional	Non-Functional
Creation	10/12/2025, 9:17pm			User	✓	
Modification				System		
Description	Provide tools for capturing, organizing, and managing training datasets for model development					
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		SFR14			
Justify why UFR12 can be completely covered by SFR14		SFR14 provides tools for dataset management, including tagging and versioning, fulfilling the need to support model development and training workflows.			
Traceability	Use cases	UC-004			
	Test cases	Not yet covered			

2. User Non-functional Requirements:

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUUR1			Type	Functional	Non-Functional
Creation	10/12/2025, 9:20pm			User		✓
Modification				System		
Description	System must be extremely easy to use for manufacturing workers with minimal training, needs to be easier than what is currently being done					
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	PUSR1, PUSR2, PUSR6, PUSR7, PUSR10				
Justify why PUUR1 can be completely **covered by PUSR1, PUSR2, PUSR6, PUSR7, PUSR10**	These requirements ensure the UI is intuitive, uses plain language, includes tutorials, and supports diverse user needs, making the system easier to use than current manual processes.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUUR2			Type	Functional	Non-Functional
Creation	10/12/2025, 9:21pm			User		✓
Modification				System		
Description	Minimize physical interaction requirements during normal operation					
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	PUSR4, PUSR5				
Justify why PUUR2 can be completely **covered by PUSR4, PUSR5**	PUSR4 enables touch-free operation, and PUSR5 automates common tasks, minimizing physical interaction during normal use.				
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUUR3			Type	Functional	Non-Functional
Creation	10/12/2025, 9:22pm			User		✓
Modification				System		
Description	Training and configuration interfaces must be accessible to non-technical staff					
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	PUSR6, PUSR7, PUSR8				
Justify why PUUR3 can be completely **covered by PUSR6, PUSR7, PUSR8**	These requirements ensure that configuration interfaces are simple, guided, and modifiable without programming, making them accessible to non-technical staff.				
Traceability	Use cases	UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUUR4			Type	Functional	Non-Functional
Creation	10/12/2025, 9:22pm			User		✓
Modification				System		
Description	Design system to accommodate workers with varying technical skills and possible disabilities					
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	PUSR9, PUSR10, PUSR11, PUSR12				
Justify why PUUR4 can be completely **covered by PUSR9, PUSR10, PUSR11, PUSR12**	These requirements ensure accessibility through visual aids, iconography, compliance standards, and input flexibility, supporting users with varying skills and disabilities.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004, UC-005			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**

Requirement #	PPUR1			Type	Functional	Non-Functional
Creation	10/12/2025, 9:23pm			User		✓
Modification				System		
Description	Process video feeds and provide feedback with minimal latency to prevent manufacturing errors					
Priority	Highest	**High ✓**	Medium	Low		Lowest
This Req. is Refined Into		PPSR1, PPSR2				
Justify why PPUR1 can be completely covered by PPSR1, PPSR2		PPSR1 ensures fast feedback (<200ms), and PPSR2 prioritizes real-time tasks, preventing manufacturing errors due to latency.				
Traceability	Use cases	UC-001, UC-002				
	Test cases	Not yet covered				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPUR2			Type	Functional	Non-Functional
Creation	10/12/2025, 9:24pm			User		✓
Modification				System		
Description	Maintain high accuracy in both barcode detection and process monitoring for aerospace quality standards					
Priority	**Highest ✓**	High	Medium	Low		Lowest
This Req. is Refined Into		PPSR3				
Justify why PPUR2 can be completely covered by PPSR3		PPSR3 enforces high barcode detection accuracy, meeting aerospace-level quality standards.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004				
	Test cases	Not yet covered				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPUR3			Type	Functional	Non-Functional
Creation	10/12/2025, 9:25pm			User		✓

Modification				System		
Description	Function effectively with lower-quality, cost-effective camera equipment				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into		PPSR5, PPSR6			
Justify why PPUR3 can be completely covered by PPSR5, PPSR6		These requirements ensure the system works with low-cost cameras and adapts to lighting, fulfilling the need for cost-effective deployment.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDUR1			Type	Functional	Non-Functional
Creation	10/12/2025, 9:25pm			User		✓
Modification				System		
Description	All processing and data storage must remain on-premises with no cloud dependencies				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		PDSR1, PDSR2			
Justify why PDUR1 can be completely covered by PDSR1, PDSR2		These requirements ensure all data and processing remain on-premises, meeting strict security and compliance needs.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PDUR2	Type	Functional	Non-Functional
Creation	10/12/2025, 9:25pm	User		✓
Modification		System		
Description	Reliable operation in industrial factory conditions with minimal downtime			

Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		PDSR3, PDSR4			
Justify why PDUR2 can be completely **covered by PDSR3, PDSR4**		PDSR3 ensures high uptime, and PDSR4 validates operation under factory conditions, supporting reliability.			
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	ODUR1		Type	Functional	Non-Functional
Creation	10/12/2025, 9:28pm		User		✓
Modification			System		
Description	Utilize open-source libraries and frameworks to minimize licensing costs				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		ODSR1, ODSR2			
Justify why ODUR1 can be completely **covered by ODSR1, ODSR2**		ODSR1 mandates open-source use, and ODSR2 documents licenses, minimizing costs and ensuring transparency.			
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOUR1		Type	Functional	Non-Functional
Creation	10/12/2025, 9:33pm		User		✓
Modification			System		
Description	Minimize hardware and deployment costs while meeting performance requirements				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		OOSR1, OOSR2			

Justify why OOUR1 can be completely covered by OOSR1, OOSR2		OOSR1 ensures performance on standard hardware, and OOSR2 allows customizable deployment, reducing hardware costs.
Traceability	Use cases	N/A
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOUR2		Type	Functional	Non-Functional
Creation	10/12/2025, 9:34pm		User		✓
Modification			System		
Description	Support flexible deployment configurations for different workstation setups				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	OOSR2, OOSR3				
Justify why OOUR2 can be completely covered by OOSR2, OOSR3	These requirements support flexible deployment and phased rollout, accommodating different workstation setups.				
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOUR3		Type	Functional	Non-Functional
Creation	10/12/2025, 9:34pm		User		✓
Modification			System		
Description	Support gradual implementation with minimal disruption to existing operations				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	OOSR3, OOSR4				
Justify why OOUR3 can be completely	OOSR3 supports gradual implementation, and OOSR4 ensures compatibility with legacy formats, minimizing disruption.				

covered by OOSR3, OOSR4		
Traceability	Use cases	N/A
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EIUR1		Type	Functional	Non-Functional
Creation	10/12/2025, 9:35pm		User		✓
Modification			System		
Description	Maintain compatibility with existing KMD Technology Solutions software interfaces				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into	EISR1, EISR2				
Justify why EIUR1 can be completely **covered by EISR1, EISR2**	EISR1 ensures API compatibility, and EISR2 maintains consistent data formats, fulfilling integration needs with KMD systems.				
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EIUR2		Type	Functional	Non-Functional
Creation	10/12/2025, 9:36pm		User		✓
Modification			System		
Description	Publish comprehensive JSON schema for scan results and system outputs				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	EISR3, EISR4				
Justify why EIUR2 can be completely **covered by EISR3, EISR4**	EISR3 defines the JSON schema, and EISR4 version-controls it, ensuring reliable and documented data exchange.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EIUR3		Type	Functional	Non-Functional
Creation	10/12/2025, 9:37pm		User		✓
Modification			System		
Description	Support integration with existing manufacturing systems and processes				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		EISR5, EISR6			
Justify why EIUR3 can be completely covered by EISR5, EISR6		EISR5 supports integration with MES and ERP tools, and EISR6 allows configuration to match factory workflows, fulfilling broader interoperability needs.			
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

3. System Functional Requirements

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Fixed-station cameras shall detect, decode, and output all barcodes or QR codes visible in a single frame				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR1			
Justify why meeting SFR1 Can contribute to the fulfillment of UFR1		SFR1 enables the system to detect and decode barcodes or QR codes in real time, directly fulfilling UFR1's need for automated barcode processing at each workstation.			
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**

Requirement #	SFR2			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Provide per-workstation configurable mapping to assign barcode spatial position to named data fields					
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR2				
Justify why meeting SFR2 Can contribute to the fulfillment of UFR2	SFR2 provides a configurable mapping system that allows barcode positions to be assigned to specific data fields, fulfilling UFR2's requirement for flexible data association.				
Traceability	Use cases	UC-001			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR3			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Recognize and extract etched/carved alphanumeric identifiers on large parts					
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	UFR3				
Justify why meeting SFR3 Can contribute to the fulfillment of UFR3	SFR3 allows the system to recognize etched or carved identifiers on large parts, fulfilling UFR3's need to track barcodes beyond paper-based formats.				
Traceability	Use cases	UC-001			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR4	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System	✓	
Description	Aggregate decoded and recognized values into a structured JSON document per scan event			

Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR1, UFR4			
Justify why meeting SFR4 Can contribute to the fulfillment of UFR1 and UFR4		SFR4 aggregates decoded barcode data into structured JSON, fulfilling UFR1's need for data formatting and UFR4's requirement for downstream system integration.			
Traceability	Use cases	UC-001			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR5			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Support real-time delivery via local REST API, local DB persistence, and keyboard-emulation mode for KMD integration					
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR9			
Justify why meeting SFR5 Can contribute to the fulfillment of UFR9		SFR5 supports integration via REST API, local database, and keyboard emulation, fulfilling UFR9's need for seamless interoperability with KMD's workflow system.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR6			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Detect and report start/completion of defined manufacturing steps and validate configured sequence					
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		UFR5			

Justify why meeting SFR6 Can contribute to the fulfillment of UFR5		SFR6 detects and validates manufacturing steps in sequence, directly fulfilling UFR5's need for process monitoring and error prevention.
Traceability	Use cases	UC-002, UC-004
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR7		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Provide per workstation UI overlay with live video, step progress indicators, visual status, and optional audio alerts for violations				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	UFR6				
Justify why meeting SFR7 Can contribute to the fulfillment of UFR6	SFR7 provides visual and audio alerts at workstations, fulfilling UFR6's requirement for immediate feedback on process status and violations.				
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR8		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Provide an in-site capture tool for non-technical users to record reference video, mark step start/end, label steps, and save training sessions with metadata				
Priority	Highest	**High**	Medium ✓	Low	Lowest
This Req. is Engineered From	UFR8				
Justify why meeting SFR8 Can contribute to the fulfillment of UFR8	SFR8 enables non-technical users to record and label training videos, fulfilling UFR8's need for an intuitive training interface.				
Traceability	Use cases	UC-003			

	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR9		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Models must tolerate natural worker variations and provide configurable sensitivity per step to reduce nuisance alerts				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	UFR8				
Justify why meeting SFR9 Can contribute to the fulfillment of UFR8	SFR9 ensures the system tolerates natural worker variations and allows sensitivity adjustments, fulfilling UFR8's need for flexible and user-friendly training.				
Traceability	Use cases	UC-002, UC-003, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR10		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	The system shall log every detected deviation from the step process including errors.				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	UFR5, UFR7				
Justify why meeting SFR10 Can contribute to the fulfillment of UFR5 and UFR7	SFR10 logs all process violations, fulfilling UFR5's need for step validation and UFR7's requirement for error tracking and review.				
Traceability	Use cases	UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**

Requirement #	SFR11			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable					
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	UFR7				
Justify why meeting SFR11 Can contribute to the fulfillment of UFR7	SFR11 provides local data storage and replay capabilities, fulfilling UFR7's need for error review and UFR10's requirement for decoupled data handling.				
Traceability	Use cases	UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR12			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields					
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	UFR4, UFR9				
Justify why meeting SFR12 Can contribute to the fulfillment of UFR4 and UFR 9	SFR12 publishes a stable JSON schema and documentation, fulfilling UFR4's need for structured output and UFR9's requirement for integration support.				
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR13			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	

Description	Provide a simulated workstation environment to replay camera feeds, inject synthetic barcodes/etched IDs, and verify API/keyboard-emulation outputs before production deployment				
Priority	Highest	High	**Medium** ✓	Low	Lowest
This Req. is Engineered From		UFR11			
Justify why meeting SFR13 Can contribute to the fulfillment of UFR11		SFR13 provides a simulated environment for testing, fulfilling UFR11's need for validation before production deployment.			
Traceability	Use cases	UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR14		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Provide tools and storage to capture, tag, version, and export labeled datasets for model training, including metadata				
Priority	Highest	High	**Medium** ✓	Low	Lowest
This Req. is Engineered From		UFR12			
Justify why meeting SFR14 Can contribute to the fulfillment of UFR12		SFR14 offers tools for dataset management, fulfilling UFR12's need to support model training and development.			
Traceability	Use cases	UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR15		Type	Functional	Non-Functional
Creation	10/27/2025, 9:35am		User		
Modification			System	✓	
Description	Fixed-station cameras shall decode all barcodes or QR codes visible in a single frame				
Priority	Highest ✓	**High**	Medium	Low	Lowest

This Req. is Engineered From		UFR1
Justify why meeting SFR1 Can contribute to the fulfillment of UFR1		SFR15 enables the system to decode all of the barcodes fulfilling part of UFR1
Traceability	Use cases	UC-001
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR16		Type	Functional	Non-Functional
Creation	10/27/2025, 9:35am		User		
Modification			System	✓	
Description	Fixed-station cameras shall output all barcodes or QR codes visible in a single frame				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR1			
Justify why meeting SFR1 Can contribute to the fulfillment of UFR1		SFR16 allows the system to output all of the barcodes to go where they need to go, partially fulfilling UFR1.			
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

4. System Non-functional Requirements

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The UI shall be intuitive and require no more than 30 minutes of training for first time users				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR8, PUUR1			
Justify why meeting PUSR1 Can contribute to the fulfillment of UFR8, PUUR1		PUSR1 ensures the UI is intuitive and quick to learn, fulfilling UFR8's need for usability and PUUR1's goal of minimizing training time.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			

	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR2			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall provide clear on-screen guidance and tooltips for all major functions				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		PUUR1			
Justify why meeting PUSR2 Can contribute to the fulfillment of PUUR1		PUSR2 provides tooltips and guidance, fulfilling PUUR1's requirement for ease of use for manufacturing workers.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR3			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The UI shall minimize the amount of effort required to complete required tasks compared to the existing process				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		UFR8			
Justify why meeting PUSR3 Can contribute to the fulfillment of UFR8		PUSR3 minimizes effort compared to current processes, fulfilling UFR8's goal of improving workflow efficiency.			
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PUSR4	Type	Functional	Non-Functional

Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall support touch-free operation through automatic detection and feedback mechanisms where possible				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR2, UFR8				
Justify why meeting PUSR4 Can contribute to the fulfillment of PUUR2, UFR8	PUSR4 enables touch-free operation, fulfilling PUUR2's goal of minimizing physical interaction and UFR8's usability needs.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR5			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall automate common workflow steps to reduce manual input from operators				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR2, UFR8				
Justify why meeting PUSR5 Can contribute to the fulfillment of PUUR2, UFR8	PUSR5 automates common tasks, fulfilling PUUR2's goal of reducing manual input and UFR8's need for intuitive operation.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR6			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon				

Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		PUUR3, PUUR1, UFR8			
Justify why meeting PUSR6 Can contribute to the fulfillment of PUUR3, PUUR1, UFR8		PUSR6 uses plain language and visual menus, fulfilling PUUR3's accessibility goal and UFR8's requirement for non-technical usability.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR7		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall include a guided setup and tutorial modes for initial configuration				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		PUUR3, PUUR1, UFR8			
Justify why meeting PUSR7 Can contribute to the fulfillment of PUUR3, PUUR1, UFR8		PUSR7 provides guided setup and tutorials, fulfilling PUUR3's need for accessible configuration and UFR8's training interface requirement.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR8		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall allow non-administrative users to modify basic settings without needing programming knowledge.				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		PUUR3, UFR8			

Justify why meeting PUSR8 Can contribute to the fulfillment of PUUR3, UFR8		PUSR8 allows basic settings to be modified without programming, fulfilling PUUR3's accessibility goal and UFR8's usability requirement.
Traceability	Use cases	UC-003
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR9		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The interface shall include support for large text and high-contrast display options				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR4, UFR8				
Justify why meeting PUSR9 Can contribute to the fulfillment of PUUR4, UFR8	PUSR9 supports large text and high contrast, fulfilling PUUR4's goal of accommodating users with visual impairments and UFR8's inclusivity requirement.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR10		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR4, PUUR1, UFR8				
Justify why meeting PUSR10 Can contribute to the fulfillment of PUUR4, PUUR1, UFR8	PUSR10 uses iconography and minimal text, fulfilling PUUR4's accessibility goal and UFR8's need for a clear, user-friendly interface.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR11			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall comply with accessibility standards				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From	PUUR4				
Justify why meeting PUSR11 Can contribute to the fulfillment of PUUR4	PUSR11 ensures compliance with accessibility standards, fulfilling PUUR4's requirement for inclusive design.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR12			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall support both mouse and keyboard input/navigation for inclusivity				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From	PUUR4				
Justify why meeting PUSR12 Can contribute to the fulfillment of PUUR4	PUSR12 supports both mouse and keyboard input, fulfilling PUUR4's goal of accommodating diverse user needs.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PPSR1	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System		✓

Description	The system shall process video input and deliver analysis feedback within 200 milliseconds				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR6			
Justify why meeting PPSR1 Can contribute to the fulfillment of UFR6		PPSR1 ensures feedback is delivered within 200ms, fulfilling UFR6's need for real-time alerts at workstations.			
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall prioritize real-time operations over background tasks to minimize latency				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR1			
Justify why meeting PPSR2 Can contribute to the fulfillment of UFR1		PPSR2 prioritizes real-time operations, fulfilling UFR1's need for fast barcode processing during manufacturing.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR3		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall achieve a barcode detection accuracy of at most one error per 100 scans				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR1			

Justify why meeting PPSR3 Can contribute to the fulfillment of UFR1		PPSR3 enforces high barcode detection accuracy, fulfilling UFR1's requirement for reliable scanning.
Traceability	Use cases	UC-001
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR4		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall detect and report process deviations with an error margin of less than 1%				
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Engineered From	UFR5				
Justify why meeting PPSR4 Can contribute to the fulfillment of UFR5	PPSR4 ensures process deviations are detected with less than 1% error, fulfilling UFR5's need for accurate step monitoring.				
Traceability	Use cases	UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR5		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain required accuracy levels using cameras with resolutions as low as 720p				
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Engineered From	UFR3				
Justify why meeting PPSR5 Can contribute to the fulfillment of UFR3	PPSR5 supports barcode detection using 720p cameras, fulfilling UFR3's need for cost-effective hardware compatibility.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR6			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall automatically adjust detection parameters for varying camera quality and lighting conditions				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR3			
Justify why meeting PPSR6 Can contribute to the fulfillment of UFR3		PPSR6 adjusts detection parameters for varying conditions, fulfilling UFR3's need for robust performance across environments.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR1			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall store all data on local servers within the facility				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From		UFR10, UFR7			
Justify why meeting PDSR1 Can contribute to the fulfillment of UFR10, UFR7		PDSR1 ensures data is stored locally, fulfilling UFR10's need for decoupled processing and UFR7's requirement for secure error logging.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR2			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓

Description	The system shall perform all processing operations using on-premises hardware without cloud reliance				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From		UFR10, UFR7			
Justify why meeting PDSR2 Can contribute to the fulfillment of UFR10, UFR7		PDSR2 enforces on-premises processing, fulfilling UFR10's need for secure data handling and UFR7's requirement for reliability.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR3			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall maintain a minimum uptime of 99% during operational hours				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From		UFR7			
Justify why meeting PDSR3 Can contribute to the fulfillment of UFR7		PDSR3 ensures 99% uptime, fulfilling UFR7's need for consistent error tracking and system availability.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR4			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall be tested for stable operation under factory environmental conditions				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From		UFR11			

Justify why meeting PDSR4 Can contribute to the fulfillment of UFR11		PDSR4 validates system operation under factory conditions, fulfilling UFR11's need for reliable pre-deployment testing.
Traceability	Use cases	UC-005
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	ODSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall use open-source libraries or frameworks unless a commercial dependency is explicitly approved				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		ODUR1			
Justify why meeting ODSR1 Can contribute to the fulfillment of ODUR1		ODSR1 mandates open-source use, fulfilling ODUR1's goal of minimizing licensing costs.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	ODSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	All open-source components shall be documented, including their licenses and version numbers				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		ODUR1			
Justify why meeting ODSR2 Can contribute to the fulfillment of ODUR1		ODSR2 documents licenses and versions, fulfilling ODUR1's requirement for transparency and cost control.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain performance benchmarks on standard hardware				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	OOUR1				
Justify why meeting OOSR1 Can contribute to the fulfillment of OOUR1	OOSR1 ensures performance on standard hardware, fulfilling OOUR1's goal of minimizing deployment costs.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The installation process shall include customizable deployment options				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	OOUR1, OOUR2				
Justify why meeting OOSR2 Can contribute to the fulfillment of OOUR1, OOUR2	OOSR2 provides customizable deployment options, fulfilling OOUR1's cost-efficiency goal and OOUR2's need for flexible setups.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	OOSR3	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System		✓

Description	The system shall support phased deployment alongside existing systems				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	OOUR2, OOUR3				
Justify why meeting OOSR3 Can contribute to the fulfillment of OOUR2, OOUR3	OOSR3 supports phased deployment, fulfilling OOUR2's flexibility goal and OOUR3's need for gradual implementation.				
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR4		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain backward compatibility with legacy data formats				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	OOUR3				
Justify why meeting OOSR4 Can contribute to the fulfillment of OOUR3	OOSR4 ensures backward compatibility, fulfilling OOUR3's goal of minimizing disruption to existing operations.				
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall integrate with all existing KMD Technology Solutions APIs and communication protocols				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	EIUR1				

Justify why meeting EISR1 Can contribute to the fulfillment of EIUR1		EISR1 ensures integration with KMD APIs, fulfilling EIUR1's requirement for software compatibility.
Traceability	Use cases	UC-001, UC-002, UC-004
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain consistent data exchange formats used by KMD				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	EIUR1				
Justify why meeting EISR2 Can contribute to the fulfillment of EIUR1	EISR2 maintains consistent data formats, fulfilling EIUR1's goal of seamless communication with KMD systems.				
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR3		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall define and publish a complete JSON schema for all output data				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	EIUR2				
Justify why meeting EISR3 Can contribute to the fulfillment of EIUR2	EISR3 defines a complete JSON schema, fulfilling EIUR2's requirement for structured output documentation.				
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR4		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The JSON schema shall be version-controlled and included in system documentation				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	EIUR2				
Justify why meeting EISR4 Can contribute to the fulfillment of EIUR2	EISR4 version-controls the schema, fulfilling EIUR2's goal of maintaining reliable and traceable data exchange.				
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR5		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall support data exchange with existing Manufacturing Execution Systems and ERP tools				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	EIUR3				
Justify why meeting EISR5 Can contribute to the fulfillment of EIUR3	EISR5 supports integration with MES and ERP tools, fulfilling EIUR3's goal of broader system interoperability.				
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR6		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓

Description	The system shall be configurable to align with existing factory workflows and data pipelines				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		EIUR3			
Justify why meeting EISR6 Can contribute to the fulfillment of EIUR3		EISR6 allows configuration to match factory workflows, fulfilling EIUR3's requirement for alignment with existing processes.			
Traceability	Use cases	UC-001, UC-002, UC-004			
	Test cases				

5. Trace Table

Project Name: AIBI CV for Manufacturing			
User Requirements		System Requirements	
Req ID	Description	Req ID	Description
UFR1	Fixed cameras at each workstation must detect, decode, and process multiple barcodes from objects presented within the camera's field of view	SFR1	Fixed-station cameras shall detect all barcodes or QR codes visible in a single frame
		SFR4	Aggregate decoded and recognized values into a structured JSON document per scan event
		PPSR2	The system shall prioritize real-time operations over background tasks to minimize latency
		PPSR3	The system shall achieve a barcode detection accuracy of at most one error per 100 scans.
		SFR15	Fixed-station cameras shall decode all barcodes or QR codes visible in a single frame
		SFR16	Fixed-station cameras shall output all barcodes or QR codes visible in a single frame
UFR2	Provide configurable mapping system to assign barcode positions to specific data fields	SFR2	Provide per-workstation configurable mapping to assign barcode spatial position to named data fields

UFR3	Identify and track QR codes or barcodes on large parts rather than sheets of paper	SFR3	Recognize and extract etched/carved alphanumeric identifiers on large parts
		PPSR5	The system shall maintain required accuracy levels using cameras with resolutions as low as 720p
		PPSR6	The system shall automatically adjust detection parameters for varying camera quality and lighting conditions
UFR4	Aggregate all decoded values into structured JSON payload for downstream system integration	SFR4	Aggregate decoded and recognized values into a structured JSON document per scan event
		SFR12	Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields
		EISR3	The system shall define and publish a complete JSON schema for all output data
		EISR4	The JSON schema shall be version-controlled and included in system documentation
UFR5	Monitor and validate that manufacturing steps are performed in correct sequence at each workstation	SFR6	Detect and report start/completion of defined manufacturing steps and validate configured sequence
		SFR10	The system shall log every detected deviation from the step process including errors.
		PPSR4	The system shall detect and report process deviations with an error margin of less than 1%
UFR6	Provide immediate visual feedback and alerts for process status and violations on terminals at workstations	SFR7	Provide per workstation UI overlay with live video, step progress indicators, visual status, and optional audio alerts for violations
		PPSR1	The system shall process video input and deliver analysis feedback within 200 milliseconds

UFR7	Log all errors caught by the system for the users to review	SFR10	The system shall log every detected deviation from the step process including errors.
		SFR11	Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable
		PDSR1	The system shall store all data on local servers within the facility
		PDSR2	The system shall perform all processing operations using on-premises hardware without cloud reliance
		PDSR3	The system shall maintain a minimum uptime of 99% during operational hours
UFR8	Provide intuitive UI for non-technical users to train the system on correct manufacturing processes	SFR8	Provide an in-site capture tool for non-technical users to record reference video, mark step start/end, label steps, and save training sessions with metadata
		SFR9	Models must tolerate natural worker variations and provide configurable sensitivity per step to reduce nuisance alerts
		PSUR1	The UI shall be intuitive and require no more than 30 minutes of training for first time users
		PSUR2	The system shall provide clear on-screen guidance and tooltips for all major functions
		PUSR3	The UI shall minimize the amount of effort required to complete required tasks compared to the existing process
		PUSR4	The system shall support touch-free operation through automatic detection and feedback mechanisms where possible

		PUSR5	The system shall automate common workflow steps to reduce manual input from operators
		PUSR6	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon
		PUSR7	The system shall include a guided setup and tutorial modes for initial configuration
		PUSR8	The system shall allow non-administrative users to modify basic settings without needing programming knowledge
		PUSR9	The interface shall include support for large text and high-contrast display options
		PUSR10	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff
UFR9	Seamless integration with existing KMD Technology Solutions workflow management system	SFR5	Support real-time delivery via local REST API, local DB persistence, and keyboard-emulation mode for KMD integration
		SFR12	Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields
UFR10	Implement intermediate data storage to decouple computer vision processing from KMD data consumption	SFR11	Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable
		PDSR1	The system shall store all data on local servers within the facility
		PDSR2	The system shall perform all processing operations using on-premises hardware without cloud reliance
UFR11	Provide simulated workstation environment for development and	SFR13	Provide a simulated workstation environment to replay camera feeds,

	validation before production deployment		inject synthetic barcodes/etched IDs, and verify API/keyboard-emulation outputs before production deployment
		PDSR4	The system shall be tested for stable operation under factory environmental conditions
UFR12	Provide tools for capturing, organizing, and managing training datasets for model development	SFR14	Provide tools and storage to capture, tag, version, and export labeled datasets for model training, including metadata
PUUR1	System must be extremely easy to use for manufacturing workers with minimal training, needs to be easier than what is currently being done	PUSR1	The UI shall be intuitive and require no more than 30 minutes of training for first time users
		PUSR2	The system shall provide clear on-screen guidance and tooltips for all major functions
		PUSR6	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon
		PUSR7	The system shall include a guided setup and tutorial modes for initial configuration
		PUSR10	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff
PUUR2	Minimize physical interaction requirements during normal operation	PUSR4	The system shall support touch-free operation through automatic detection and feedback mechanisms where possible
		PUSR5	The system shall automate common workflow steps to reduce manual input from operators
PUUR3	Training and configuration interfaces must be accessible to non-technical staff	PUSR6	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon
		PUSR7	The system shall include a guided setup and tutorial modes for initial configuration

		PUSR8	The system shall allow non-administrative users to modify basic settings without needing programming knowledge.
PUUR4	Design system to accommodate workers with varying technical skills and possible disabilities	PUSR9	The interface shall include support for large text and high-contrast display options
		PUSR10	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff
		PUSR11	The system shall comply with accessibility standards
		PUSR12	The system shall support both mouse and keyboard input/navigation for inclusivity
PPUR1	Process video feeds and provide feedback with minimal latency to prevent manufacturing errors	PPSR1	The system shall process video input and deliver analysis feedback within 200 milliseconds
		PPSR2	The system shall prioritize real-time operations over background tasks to minimize latency
PPUR2	Maintain high accuracy in both barcode detection and process monitoring for aerospace quality standards	PPSR3	The system shall achieve a barcode detection accuracy of at most one error per 100 scans
PPUR3	Function effectively with lower-quality, cost-effective camera equipment	PPSR5	The system shall maintain required accuracy levels using cameras with resolutions as low as 720p
		PPSR6	The system shall automatically adjust detection parameters for varying camera quality and lighting condition
PDUR1	All processing and data storage must remain on-premises with no cloud dependencies or external services	PDSR1	The system shall store all data on local servers within the facility
		PDSR2	The system shall perform all processing operations using on-premises hardware without cloud reliance

PDUR2	Reliable operation in industrial factory conditions with minimal downtime	PDSR3	The system shall maintain a minimum uptime of 99% during operational hours
		PDSR4	The system shall be tested for stable operation under factory environmental conditions
ODUR1	Utilize open-source libraries and frameworks to minimize licensing costs	ODSR1	The system shall use open-source libraries or frameworks unless a commercial dependency is explicitly approved
		ODSR2	All open-source components shall be documented, including their licenses and version numbers
OOUR1	Minimize hardware and deployment costs while meeting performance requirements	OOSR1	The system shall maintain performance benchmarks on standard hardware
		OOSR2	The installation process shall include customizable deployment options
OOUR2	Support flexible deployment configurations for different workstation setups	OOSR2	The installation process shall include customizable deployment options
		OOSR3	The system shall support phased deployment alongside existing systems
OOUR3	Support gradual implementation with minimal disruption to existing operations	OOSR3	The system shall support phased deployment alongside existing systems
		OOSR4	The system shall maintain backward compatibility with legacy data formats
EIUR1	Maintain compatibility with existing KMD Technology Solutions software interfaces	EISR1	The system shall integrate with all existing KMD Technology Solutions APIs and communication protocols
		EISR2	The system shall maintain consistent data exchange formats used by KMD
EIUR2	Publish comprehensive JSON schema for scan results and system outputs	EISR3	The system shall define and publish a complete JSON schema for all output data
		EISR4	The JSON schema shall be version-controlled and included in system documentation

EIUR3	Support integration with existing manufacturing systems and processes	EISR5	The system shall support data exchange with existing Manufacturing Execution Systems and ERP tools
		EISR6	The system shall be configurable to align with existing factory workflows and data pipelines